In-kernel	**Kernel bypass / userspace**
eBPF kernel programs [3, 4, 10]	DPDK [1, 6, 9, 10, 11, 12, 13]
XDP (native/generic) [3, 10]	Snabb [11]
SK_MSG [7]	PF_RING ZC [1]
Linux network stack [3, 10, 11]	netmap [1]
AF_PACKET, PACKET_MMAP [9, 12, 13]	AF_XDP [2, 8, 9, 10, 12, 13]
	NIC-vendor bypass (mlx5 Direct Verbs, ef_vi) [12, 13]
	uXDP over DPDK/AF_XDP [5]
	VPP over DPDK [11]

Paper	Approaches	Workload	Metrics	Comparison basis
Packet IO (2015) [1]	DPDK, netmap, PF_RING ZC	Forwarding, fixed CPU cost, random accesses to 1–512 MB working sets	Packet rate, cycles/packet, batch/cache effects, latency at 99% line rate	Same CPU, 10-Gbit NICs and forwarding algorithm.
AF_XDP / OpenNIC (2025) [2]	Mixed-mode AF_XDP zero-copy, DPDK on OpenNIC	Packet drop and L2 MAC-swap forwarding	Rate vs. packet size and queue count	Same FPGA, different drivers and benchmark tools.
XDP (2018) [3]	Native XDP, DPDK `testpmd`, Linux stack	64-byte drop and MAC-swap forwarding, core scaling	Rate vs. cores, CPU use vs. load, forwarding latency	Same testbed. RSS selects cores. Linux stack only in drop/CPU tests.
Linux survey (2022) [4]	Hardware, software, virtualisation	Offload, RDMA, GPU, zero-copy, bypass, XDP, virtual NICs	Resource efficiency, packet rate, security, flexibility	Literature review, no shared benchmark/testbed.
uXDP (2025) [5]	Native/generic XDP, userspace eBPF via DPDK/AF_XDP	Counters, filtering, load balancing, Katran, MAC-swap echo (`xdp_tx`)	NF throughput, unloaded RTT, compiler/inlining effects	Same programs/hardware, one core. Mode comparison: optimised source LLVM IR.
Metronome (2023) [6]	DPDK busy polling, adaptive sleep/wake, native XDP	L3 `l3fwd`, load changes, concurrent CPU workloads	Rate, loss, CPU use, latency vs. sleep interval, power	DPDK/Metronome: same `l3fwd`. Different thread/core allocations.
MiddleNet (2023) [7]	DPDK (PMD, RTE_RING), AF_XDP, eBPF SK_MSG, SR-IOV	L3 routing, L2 forwarding, HTTP proxy, URL rewrite, chain scaling	MLFR, CPU and latency at MLFR, HTTP request rate/latency, copy and interrupt overhead	Same workloads/testbed. Five 60-s L2/L3 runs. Data paths and CPU budgets differ.
Edge data centers (2023) [8]	Native XDP, AF_XDP (busy loop, syscall, `poll()`), XDP with AF_XDP sockets	Drop, transit, local traffic, CPU/memory scaling, firewall, load balancer, memcached	Throughput, CPU by execution context, LLC hits/misses	40-Gbit testbed, usually one core. 10 runs (mean/SD). Drop/transit loss below 0.1%.
UPF I/O modes (2026) [9]	AF_PACKET, AF_XDP zero-copy, CNDP, DPDK PMD as BESS ports behind `RecvPackets` and `SendPackets`	5G UPF pipeline (GTP-U, PDR/FAR/QER), 64–1518 B, 1–8 workers, 24-h soak	Non-drop rate (RFC 2544), in-pipeline latency percentiles, per-core rate, worker scaling	Same host, NIC, kernel and UPF image, only the port driver changes. Different generators per mode. AF_XDP/CNDP capped at two workers.
OVS dataplane (2021) [10]	OVS kernel module, OVS-DPDK, OVS-AF_XDP, eBPF/XDP variants	NSX rule set with TCP, 64-byte P2P/PVP/PCP forwarding, `netperf` RR, 1–6 queues	TCP Gbps, lossless Mpps, CPU by context, P50/P90/P99 latency	Same switch and testbed, I/O backend swapped. Kernel path uses RSS over all cores, userspace paths pinned PMD threads.
Inter-container bypass (2020) [11]	Linux bridge/sockets, DPDK basicfwd, OVS-DPDK, VPP, Snabb, SR-IOV	Container pairs on one or two hosts, 64–1500 B, bursts 4–256, 1–8 pairs	Max receive rate, total throughput vs. pairs, mean round-trip latency	Same hosts and benchmark apps (`nfv-testperf`), polling, fixed CPU clock. Switching logic differs per switch.
packetio (2026) [12]	AF_PACKET, AF_XDP, DPDK, mlx5 Direct Verbs behind one Go API	Transmit, receive, L3 LPM forward, 64 B at 100 G	Mpps per core, cores to line rate incl. softirq, median RTT (mlx5 only)	Same sweep program, backend by build tag. No XDP, no partial load, no energy.
ABTRDA3 (2026) [13]	ef_vi, DPDK, AF_XDP, verbs, PACKET_MMAP behind C++20 ring concepts	MAC-swap reflector, one frame in flight, 24-h soak	RTT median to P99.999	Same loop, transport by config file. Latency only.

Comparison of Frameworks for High-Performance Packet IO [1]

DPDK and PF_RING ZC outperform netmap, and the CPU cycles available per packet bound throughput.

DPDK and PF_RING ZC process packets in user space using zero-copy. The Linux stack handles packets in kernel space with security and robustness checks. netmap combines userspace processing with kernel buffer checks and packet transfer through system calls. DPDK and PF_RING ZC outperform netmap in most tested forwarding scenarios. Available CPU cycles limit throughput.

Metrics

- *CPU cycles per packet.* More cycles per packet reduce throughput (Figure 2a). C_{task} is the application's processing cost per packet.
- *Cache size vs. throughput.* Throughput drops and cache misses rise when the working set reaches the CPU's 4 MB L3 cache (Figures 3a and 3b).
- *Batch size, throughput and latency.* Packets per NIC batch affect throughput and CPU cycles. Larger batches increase queueing delay. Very small batches sharply reduce throughput. netmap needs large batches for high throughput. Latency is measured at 99% of line rate.

Workload

- *Simulated task* C_{task}. Fixed CPU cycles per packet simulate application work.
- *Memory access.* One random memory access per packet, with 1–512 MB working sets.

Performance Implications at the Intersection of AF_XDP and Programmable NICs [2]

DPDK beats mixed-mode AF_XDP only for 64-byte packets, while both perform nearly equally for larger packets.

The OpenNIC driver adds eBPF/XDP loading and AF_XDP zero-copy. Without driver support, AF_XDP uses copy mode. Focus: splitting AF_XDP functionality across the FPGA, driver and kernel.

Processing paths and motivation

- *Native XDP.* XDP runs in the kernel's NIC driver. Packets cross PCIe into host memory before XDP processes them. High packet rates can exhaust host CPU capacity.
- *NIC offloading.* NIC-side filtering or processing can reduce host CPU work. Programmable hardware and hardware development are costly. AF_XDP hardware features consume FPGA resources otherwise available to application logic.
- *Studied hardware/kernel split.* Kernel eBPF selects the receiving AF_XDP socket. Software redirection reduces hardware flow-steering requirements but adds host processing. Further parameters: TX NAPI budget and NAPI placement on CPU cores.

AF_XDP mixed mode shares the NIC between bypass applications and traditional Linux sockets. Exclusive DPDK deployments assign the NIC to one application.

Metrics

- *Throughput vs. packet size.* Small packets require higher packet rates at the same data rate. DPDK outperforms mixed-mode AF_XDP (XSK) for 64-byte packets. AF_XDP and DPDK perform nearly equally for larger packets (Figures 7 and 8).

Workload

- *Packet drop.* Receive and drop packets.
- *L2 swap and forward.* Swap source/destination MAC addresses and forward.

The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel [3]

XDP drops about five times more packets per core than the Linux stack with load-proportional CPU use, while DPDK stays faster at 100 % CPU.

XDP executes eBPF in the receive path before the Linux network stack. General-purpose stacks prioritise flexibility over per-packet cost. XDP reduces this overhead while preserving application isolation and kernel security. DPDK remains faster in some tested scenarios.

- Cooperates with the regular network stack.
- Remains transparent to host applications.
- CPU usage scales with load without exclusively reserved cores.

Metrics

- *Packet drop rate vs. number of cores.* Minimal drop processing measures system overhead and gives an upper bound on application throughput (Figure 3). DPDK and XDP scale approximately linearly with the number of cores until they approach the PCI bus limit. Single-core XDP achieves about five times the drop rate of the Linux stack in raw mode.
- *CPU usage vs. offered packet load.* Single-core drop test (Figure 4): DPDK busy polling uses 100% CPU. XDP and Linux CPU usage scale with offered load.
- *Packet forwarding throughput and latency.* Throughput vs. core count (Figure 5). Latency at low and high packet rates (Table 1).

Workload

- *Minimum-sized packets.* 64-byte packets maximise packet rate at a given line rate.
- *Packet drop.* Receive and drop without application processing.
- *L2 swap and forward.* Swap source/destination MAC addresses and forward. Minimal rewriting gives an upper bound on forwarding throughput.

Comparison basis

Same testbed and basic packet tasks. DPDK uses `testpmd`. Hardware Receive Side Scaling (RSS) steers XDP/Linux traffic to the selected cores. The Linux stack appears in drop and CPU tests. Minimal forwarding excludes it because it requires bridging or routing lookups.

A survey on accelerating technologies for fast network packet processing in Linux environments [4]

Survey that classifies accelerators by resource use, packet rate, security and flexibility, without a common benchmark or performance ranking.

Taxonomy of Linux packet-processing bottlenecks and accelerators: hardware, software and virtualisation (Figure 2).

Metrics

- *Host resource usage and packet rate.* Published CPU usage and packet rates. Continuous polling trades CPU occupation at low load for higher throughput.
- *Security and flexibility.* Kernel isolation, programmability, hardware dependencies and expandability.

Workload

- *Hardware acceleration.* TCP offload engines, programmable NICs, RDMA and GPU packet processing.
- *Software acceleration.* Zero-copy, kernel bypass, in-kernel processing and userspace TCP. Examples: netmap, DPDK, PF_RING ZC and XDP/eBPF (in-kernel).
- *Virtualised networking.* NIC passthrough, SR-IOV, VirtIO/VHost and userspace processing (VPP).

Comparison basis

Literature synthesis without a common benchmark or testbed. The survey's Table 2 maps studies to the four criteria. Supports technology selection, with no overall performance ranking.

---

uXDP: Frictionless XDP Deployments in Userspace [5]

DPDK-mode uXDP is fastest, up to 3.3× in-kernel XDP, but the figures mix I/O-path and compiler effects.

uXDP runs unchanged XDP/eBPF programs in userspace via DPDK or AF_XDP. It extends `bpftime`, retaining eBPF verification and existing control-plane compatibility. Optimisations: helper/map inlining and SIMD instructions. DPDK mode avoids interrupt-based retrieval and enables execution where kernel eBPF is unavailable.

Deployment modes

- *DPDK mode.* Userspace RX/TX rings and packet processing bypass the kernel network stack. Kernel-bound traffic requires reinjection.
- *AF_XDP mode.* Kernel XDP redirects packets to an AF_XDP socket for userspace eBPF processing. Shared RX/TX rings and packet buffers avoid per-packet copies in zero-copy mode.

In AF_XDP mode, kernel XDP can pass selected packets directly to the Linux stack before userspace processing. After uXDP processing, the evaluated version supports only `XDP_DROP` and `XDP_TX`. Kernel reinjection for `XDP_PASS` is future work (Section 7).

Metrics

- *Throughput across deployment modes.* Modes in Figure 4: DPDK (`dpdk_llvm`), AF_XDP (`afxdp_llvm`), native XDP (`drv_mode`), generic XDP (`skb_mode`). Up to 3.3× throughput over in-kernel execution and up to 40% higher throughput for Meta's Katran. DPDK mode is fastest across tested functions. AF_XDP mode can exceed native XDP for complex functions. Generic XDP is slowest.
- *Unloaded round-trip latency.* `xdp_tx` MAC-swap RTT (Figure 5): DPDK mode has the lowest latency, AF_XDP mode slightly higher latency than native XDP.

Workload

- *XDP network functions.* Packet counters, packet-length statistics, filtering, simple load balancing and Katran, with different helper/map combinations.
- *MAC swap and transmit.* `xdp_tx`: swap source/destination MAC addresses and transmit back (latency benchmark).

Comparison basis

Same programs and testbed, one CPU core. 64-byte TCP or 128-byte ICMP packets. Existing eBPF binaries run without recompilation. Figure 4 uses optimised source LLVM IR: results combine deployment-mode and compiler effects.

Metronome: Adaptive and Precise Intermittent Packet Retrieval in DPDK [6]

Adaptive sleep/wake keeps DPDK's throughput and loss while trading CPU time and power against buffering latency.

DPDK busy polling occupies reserved cores at 100% even at low traffic, consuming CPU time and power. Metronome adapts sleep/wake intervals to packet load. Alternating threads release CPU time for other applications. Like XDP, it targets load-proportional CPU utilisation.

Metrics

- *Latency and CPU usage vs. sleep interval.* Target sleep interval varied at 5 and 10 Gbps (Figure 5). Longer sleep intervals reduce CPU usage but increase packet buffering delay. Adaptive tuning targets a configurable latency as load changes.
- *Latency vs. number of threads.* More threads sharing a receive queue can increase latency and variance through coordination and longer sleep periods (Figures 7 and 8).
- *Power consumption.* Sleeping can reduce polling power costs. Savings depend on traffic load and CPU power governor.
- *Throughput and packet loss.* Tuned Metronome achieves comparable throughput and no substantial packet-loss difference from standard DPDK.

Workload

- *L3 forwarding.* DPDK `l3fwd` with changing traffic rates.
- *CPU sharing.* CPU-intensive applications run alongside packet I/O to use released CPU time.

Comparison basis

DPDK/Metronome: same forwarding application and hardware. Metronome typically uses three threads, DPDK one polling thread per receive queue. Native XDP comparison: `xdp_router_ipv4` (Section V-D). Core counts and frame sizes differ in that comparison.

MiddleNet: A Unified, High-Performance NFV and Middlebox Framework With eBPF and DPDK [7]

DPDK paths win for small packets and event-driven eBPF paths use less CPU for large ones, but the comparison covers whole architectures rather than isolated I/O paths.

MiddleNet combines L2/L3 network functions and L4/L7 middleboxes on one host. L2/L3 uses DPDK kernel bypass for zero-copy delivery and high packet rates. L4/L7 combines event-driven eBPF with the kernel protocol stack. SR-IOV allows both paths to coexist.

Separate NFV/middlebox platforms can place functions on different nodes. Service function chaining (SFC) transfers can add interrupts, copies, context switches, protocol processing and serialization/deserialization. Overhead grows with chain length and can reduce throughput. MiddleNet stores packet data in shared memory and passes descriptors between functions on the same host, avoiding repeated payload copies.

Shared-memory communication

NIC access: DPDK PMD or AF_XDP. Descriptor transfer between functions: RTE_RING or SK_MSG. L4/L7 uses sockets and a userspace message broker with the kernel protocol stack.

- *Event-driven eBPF.* Negligible overhead without events. Programs have size limits and must run to completion. Interrupts and context switches limit performance at high packet rates.
- *Polling-based DPDK.* PMD and RTE_RING avoid interrupts for retrieval and descriptor transfer. Continuous polling occupies CPU cores. This path provides zero-copy, interrupt-free chaining.

Metrics

- *Packet rate, CPU usage and latency.* Maximum loss-free rate (MLFR), CPU utilisation and end-to-end latency at MLFR vs. packet size (Figure 7). DPDK performs better for small packets. Event-driven variants use less CPU for larger packets.
- *Application performance.* HTTP requests/s, response latency and CPU usage vs. client concurrency (Figure 10).
- *Communication overhead.* Data copies, interrupts and context switches along the chain.

Workload

- *L2/L3 chain.* L3 routing followed by L2 forwarding updates IP/MAC addresses. DPDK: PMD and RTE_RING. eBPF: SK_MSG with AF_XDP polling or interrupts.
- *L4/L7 chain.* HTTP reverse-proxy load balancing followed by URL rewriting. Chain length and compute demand varied separately.

Comparison basis

Same testbed and comparable chains, with OVS and kernel-bridge baselines. L2/L3: mean of five 60-second runs. Results compare complete architectures with different data paths, communication mechanisms and CPU budgets.

Comparing User Space and In-Kernel Packet Processing for Edge Data Centers [8]

Native XDP wins for cache-resident dropping and syscall-driven AF_XDP for forwarding and memory-heavy processing, with latency left unmeasured.

Native XDP vs. zero-copy AF_XDP on edge servers sharing resources with applications. Kernel-provided XDP and AF_XDP, no custom modules. AF_XDP modes: busy loop, syscall-triggered driver polling and `poll()`. Additional modes: XDP processing with AF_XDP sockets enabled.

Metrics

- *Throughput.* RFC2544-based input-rate adjustment until drop/transit loss is below 0.1%. XDP performs well for cache-resident dropping. Syscall-driven AF_XDP benefits forwarding and memory-intensive processing.
- *CPU usage.* Userspace, system-call and software-interrupt CPU usage.
- *Cache and memory behaviour.* LLC load/store hits and misses vs. working set and processing cost. Latency evaluation is left to future work.

Workload

- *Traffic classes.* Drop, pass-through to remote destinations, local delivery through the kernel stack.
- *Processing and memory demand.* Minimal MAC swap isolates packet I/O overhead. Repeated checksums add CPU work. Random accesses to growing arrays add memory demand.
- *Application examples.* Firewall, load balancer, local memcached and mixed traffic.

Comparison basis

Same 40-Gbps testbed, usually one CPU core. Ten repetitions, mean and standard deviation. Results depend on traffic class, memory demand and driver scheduling mode.

Comparing UPF Dataplane I/O Modes in a Cloud-Native 5G Core: AF_PACKET, AF_XDP, CNDP, and DPDK on SD-Core BESS-UPF [9]

One UPF pipeline, four I/O backends behind the same BESS port primitives: at two workers AF_XDP, CNDP and DPDK are within measurement noise and AF_XDP leads per core, DPDK's lead is a scaling-ceiling effect, and AF_PACKET is an order of magnitude slower.

Earlier work compares I/O modes in pairs, most often AF_XDP against DPDK, or with synthetic generators, so UPF operators have to combine results from different testbeds and applications before they can decide. Here one open-source 5G user plane function (SD-Core BESS-UPF) runs under four I/O modes and only the port from which packets are received changes, so the I/O backend is the only variable. Measured are throughput, latency, CPU use and stability; deployment requirements, operational pitfalls and a decision guide are reported next to the measurements (Sections 3, 7 and 9).

Port abstraction

- *BESS port as black box.* The Berkeley Extensible Software Switch hides the port from the userspace pipeline: the UPF only calls `RecvPackets` and `SendPackets` on packet batches. A configuration file selects driver (for example the DPDK poll-mode driver), ring and kernel involvement; all of it stays invisible to the UPF code.
- *Four modes.* AF_PACKET and AF_XDP through DPDK poll-mode drivers, CNDP through its own port type, DPDK through a native PMD over `vfio-pci`.
- *CNDP.* A userspace abstraction on top of AF_XDP with a DPDK-like programming model (mempools, ports) but without DPDK's hugepage and VFIO requirements.

Metrics

- *Throughput.* Non-drop rate per RFC 2544 over 64–1518 B (Figure 18, Tables 3 and 4). At 64 B: CNDP 5.52 and AF_XDP 6.47 Mpps with two workers, DPDK 10.30 Mpps with four and 13.09 Mpps with eight; AF_PACKET 0.25 Mpps at 86 B.
- *Latency and jitter.* In-pipeline probes give mean, P50, P99 and P99.9 (Table 5): DPDK 8.1 µs mean, CNDP and AF_XDP 13–14 µs. A second AF_XDP worker adds P99 jitter through contention on the kernel redirect path (Table 6).
- *CPU and core scaling.* Per-core rate and scaling efficiency (Table 7): AF_XDP 3.69, DPDK 2.99, CNDP 2.65 Mpps per core. A per-netdev AF_XDP socket limit pins CNDP and AF_XDP at two workers.

Workload

- *UPF pipeline.* GTP-U decapsulation, PDR lookup, QER metering and FAR action with a single rule set.
- *Synthetic and real traffic.* `testpmd` and kernel `pktgen` for I/O ceilings; a 24-hour soak and a real-RAN run validate CNDP only.

Comparison basis

Same host, NIC (Intel XXV710), real-time kernel, Kubernetes cluster and UPF image; the port driver is the only variable (Table 2). Caveats: a different load generator per mode, traffic hairpinned inside the NIC because of 1-GbE optics, a NIC receive path that caps every mode near 3 Mpps per core, no energy measurement, no in-kernel XDP path. BESS adds its own scheduler and module pipeline above the port, so the measured path is BESS plus port, not the port alone.

Revisiting the Open vSwitch Dataplane Ten Years Later [10]

DPDK stays ahead of AF_XDP on physical and VM paths at lower CPU cost, the kernel datapath keeps up only by spending about eight cores through RSS, and AF_XDP wins container forwarding via XDP redirect.

Open vSwitch (OVS) moves from a kernel-module datapath to userspace over AF_XDP, with DPDK as the performance reference. The motivation is maintainability (Section 2), the paper closes with two years of lessons learned (Section 6). An eBPF datapath at the `tc` hook was rejected as 10–20 % slower than the kernel module (Figure 2).

Metrics

- *Forwarding rate and CPU use.* Maximum lossless rate for 64-byte packets with 1 and 1,000 flows; CPU in hyperthread units split into system, softirq, guest and user (Figure 9, Table 4). With vhostuser, AF_XDP is about 30 % slower than DPDK.
- *Latency.* `netperf` TCP_RR P50/P90/P99 (Figures 10 and 11): inter-host DPDK 36/38/45 µs, AF_XDP 39/41/53, kernel 58/68/94; between containers DPDK 81/136/241 versus about 15/16/20 for kernel and AF_XDP.
- *Scaling and XDP cost.* Throughput over 1–6 queues at 25 Gbps (Figure 12). Single-core XDP rate falls from 14 to 4.7 Mpps as program complexity grows (Table 5).

Workload

- *Production rule set.* About 103,000 NSX OpenFlow rules with Geneve tunnels and connection tracking, TCP via `iperf` for VM and container scenarios (Figure 8).
- *Raw forwarding.* TRex 64-byte traffic through physical-to-physical, physical-VM-physical and physical-container-physical loops.

Comparison basis

OVS 2.14 with swapped I/O backend on one testbed. The kernel datapath spreads load over all 16 hyperthreads via RSS, userspace datapaths use pinned PMD threads. Virtual device type (tap versus vhostuser) and the missing AF_XDP checksum offload (estimated) shift results. DPDK is absent from the NSX scenario.

Comparative Evaluation of Kernel Bypass Mechanisms for High-performance Inter-container Communications [11]

Every bypass path beats kernel sockets by over four times, SR-IOV leads throughput and large-burst latency, VPP nearly matches raw DPDK forwarding, and Snabb trails on both throughput and latency.

An open-source framework (`nfv-testperf`) benchmarks virtual switches between LXC containers. Compared: Linux bridge with POSIX sockets, the DPDK basic forwarding sample, OVS-DPDK, VPP, Snabb as a learning switch, and SR-IOV with the NIC's embedded switch. Benchmark applications poll and use one API over POSIX or DPDK.

Metrics

- *Throughput.* Maximum receive rate in Mpps and Gbps versus packet size (Figure 4). Kernel sockets reach 0.34–0.44 Mpps, DPDK-based switches at least 2 Mpps (Table 2). Burst sizes 32–256 did not change throughput.
- *Scalability.* Total throughput for 1–8 container pairs on one or two hosts (Figures 4e–4h).
- *Latency.* Mean round-trip time at low rate versus packet and burst size (Figure 5). Only virtio-based switches reach single-digit microseconds locally; Snabb stays above 60 µs.

Workload

- *Sender/receiver pairs.* Unidirectional synthetic traffic, 64–1500-byte packets, 1–20 Mpps offered, one-minute runs.
- *Client/server pairs.* Echoed bursts of 4–128 packets for round-trip latency.

Comparison basis

Two identical hosts (Xeon E5-2640 v4, Intel X710 10 GbE), frequency scaling and hyperthreading off; DPDK 19.05, OVS 2.11, Snabb 2019.01, VPP 19.08. Switching logic differs: the DPDK sample does no switching, Snabb's L2 component is its bottleneck, SR-IOV switches in hardware. Snabb runs single-host only; CPU cores per switch are not varied.

Userspace I/O abstractions: packetio and ABTRDA3 [12, 13]

Both expose one API over several ways to reach the NIC, so a generic userspace I/O interface is state of practice; the frame idea goes further by including the in-kernel path, which leaves the overlap large for DPDK and AF_XDP and nil for XDP and Snabb.

Two open-source projects from 2026 without peer review. *packetio* (Go) hides as much setup complexity as possible behind one API that exposes AF_PACKET, AF_XDP, DPDK and mlx5 Direct Verbs; the same sweep program (transmit, receive, L3 forward) runs on all four backends. *ABTRDA3* (C++20) follows the same concept because every kernel-bypass framework has a different setup ritual: the transport is a template parameter behind `TxRing`/`RxRing` concepts, chosen in a TOML file, over ef_vi, DPDK, AF_XDP, verbs and PACKET_MMAP.

Metrics

- *packetio.* Mpps at one core and cores to line rate on an AMD EPYC 9275F with a ConnectX-6 Dx at 100 G, 64-byte frames, measured September 2026: forwarding per core mlx5 29.3, DPDK 22.6, AF_XDP 17.5 (two cores); AF_PACKET collapses under load. Cores are counted machine-wide including softirq work. Latency only as medians on mlx5.
- *ABTRDA3.* 24-hour loopback RTT percentiles up to P99.999 on X2522, ConnectX-4 Lx and XXV710 (ef_vi 1.87, verbs 2.43, DPDK 3.51 µs median). No throughput, CPU or energy.

Comparison basis

Same program per project, backend swapped by build tag or configuration file. No in-kernel XDP, no Snabb, no empty-frame calibration, no partial-load sweep, no energy. Design references for the frame interface (opaque packet handle, backend by configuration, CPU including kernel work), not measurement studies.

References

[1] S. Gallenmüller et al. *Comparison of Frameworks for High-Performance Packet IO*. ANCS, 2015. PDF.

[2] M. Molè et al. *Performance Implications at the Intersection of AF_XDP and Programmable NICs*. eBPF Workshop, 2025. DOI.

[3] T. Høiland-Jørgensen et al. *The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel*. CoNEXT, 2018. DOI.

[4] E. Freitas et al. *A survey on accelerating technologies for fast network packet processing in Linux environments*. Computer Communications, 2022. DOI.

[5] Y. Zheng, P. Gavriil and M. Kogias. *uXDP: Frictionless XDP Deployments in Userspace*. eBPF Workshop, 2025. DOI.

[6] M. Faltelli et al. *Metronome: Adaptive and Precise Intermittent Packet Retrieval in DPDK*. IEEE/ACM Transactions on Networking, 2023 (journal edition). DOI.

[7] S. Qi et al. *MiddleNet: A Unified, High-Performance NFV and Middlebox Framework With eBPF and DPDK*. IEEE Transactions on Network and Service Management, 2023. DOI.

[8] F. Parola, R. Procopio, R. Querio and F. Risso. *Comparing User Space and In-Kernel Packet Processing for Edge Data Centers*. ACM SIGCOMM Computer Communication Review, 2023. DOI.

[9] S. Valia, N. Rajput, A. Gairola and V. Rathi. *Comparing UPF Dataplane I/O Modes in a Cloud-Native 5G Core: AF_PACKET, AF_XDP, CNDP, and DPDK on SD-Core BESS-UPF*. arXiv:2608.14589, 2026. arXiv.

[10] W. Tu, Y.-H. Wei, G. Antichi and B. Pfaff. *Revisiting the Open vSwitch Dataplane Ten Years Later*. SIGCOMM, 2021. DOI.

[11] G. Ara, T. Cucinotta, L. Abeni and C. Vitucci. *Comparative Evaluation of Kernel Bypass Mechanisms for High-performance Inter-container Communications*. CLOSER, 2020. DOI.

[12] A. Toonk. *packetio: one API over four ways to reach the NIC*. GitHub repository and blog, 2026. Code, Blog.

[13] A. Sherjil. *ABTRDA3: C++20 kernel-bypass networking library with 24-hour tail-latency benchmarks*. GitHub repository, 2026. Code.